SECP3623-02 : DATABASE PROGRAMMING

SEMESTER 1 2025/2026

PROJECT II

THEME: CAFE

LECTURER: Dr. Shamini Raja Kumaran

GROUP MEMBERS

	NAME	MATRIC NUMBER
1	MUHAMMAD MUKHRITZ AL IMAN BIN MOHD RAFFI	A23CS0250
2	MUHAMMAD NAIM BIN ABDULLAH	A23CS0134
3	AHMAD ADIB ZIKRI BIN A.MAZLAM	A23CS0205
4	MUHAMMAD SYAHMI FARIS BIN RUSLI	A23CS0138

Link Presentation Video

Table Of Content

1.0 Introduction 3

2.0 NoSQL Data Model 6

2.1 Collections Overview and Explanation 6

2.2 Example JSON Document 7

2.4 Data types Used 9

3.0 Basic MongoDB Operations (CRUD) 10

4.0 Advanced MongoDB Operations 21

4.1 Query Performance Discussion 24

4.2 Challenges encountered 25

5.0 Security & Limitations 26

6.0 Teamwork 27

7.0 Conclusion 28

1.0 Introduction

This project discusses developing a cafe ordering system using a MongoDB as the database. The system represents the common daily operations of a cafe in a realistic way. It manages 3 main components which are menu items, customers and orders.

The system stores information about food and drinks such as price, availability and category. It also keeps customer information including membership type and reward points. In addition, every order that are been place by customer will be recorded, including the items purchased and payment status

MongoDB is used for databases because it is suitable for handling flexible and structured data. The system applies both embedded document and references to show relationship between collections in an efficient way

The cafe domain is chosen because it is easy to understand, real life simulation, and suitable for demonstrating how a NoSQL database is used in real business operations such as customer management, menu control and sales transactions.

The objective of this project are :

1. To design and build a complete MongoDB database model for a cafe ordering systems
2. To apply the main concepts of document-based database, including:
    - Collections
    - Embedded document
    - References between collections
    - Array and nested objects
3. To demonstrate how real world data can be modeled effectively using MongoDB
4. To perform a basic MongoDB operations (CRUD) such as :
    - Insert
    - Find
    - Update
    - Delete
5. To show how relationships between collections can be created without using traditional foreign keys, by using logical references.

Team member and roles :

	NAME	Roles
1	MUHAMMAD MUKHRITZ AL IMAN BIN MOHD RAFFI	Documentation, Security & Presentation Lead
2	MUHAMMAD NAIM BIN ABDULLAH	Data Modelling & Domain Lead
3	AHMAD ADIB ZIKRI BIN A.MAZLAM	Indexing, Aggregation & Performance Lead
4	MUHAMMAD SYAHMI FARIS BIN RUSLI	CRUD & Query Operations Lead

2.0 NoSQL Data Model

This project uses MongoDB, a document-oriented NoSQL database, to model a cafe ordering system. The data has been organised into 3 main collections: menu_item, customers, and orders. The design appropriate use of embedding and referencing to optimise performance and reflect real-world relationships.

2.1 Collections Overview and Explanation

1. Menu_Items collection

This collection stores all menu items offered by the cafe. It acts as a master data collection and contains information such as item name, category, price, availability and tags.

Purpose:

- Maintain a list of products sold by the cafe
- Allow menu updates without affecting past orders
- Support filtering by category or availability

2. Customers collection

This collection stores customer information. Each customer has an embedded membership sub-document that contains membership type and loyalty points.

Purpose:

- Store customer identity and contact information
- Track membership level and reward points
- Support customer-based queries and order references

3. Order collection

This collection stores transaction records. Each document represents one order placed by a customer. It references the customer and embeds ordered items and payment information

Purpose:

- Record cafe transactions
- Track order status and payment information
- Preserve historical records data even if menu price change

2.2 Example JSON Document

Data Collection	Example

menu_items	{
	"itemCode": "BV001",
	"name": "Cappuccino",
	"category": "Beverage",
	"price": 8.50,
	"available": true,
	"tags": ["coffee", "hot"],
	"createdAt": { "$date": "2025-01-01T00:00:00Z" }
	}

customers	{
	"customerId": "CUST001",
	"name": "Aiman Hakim",
	"phone": "0123456789",
	"membership": {
	"type": "Student",
	"points": 120

	}, "joinedAt": { "$date": "2024-12-01T00:00:00Z" } }

orders	{
	"orderId": "ORD001",
	"customerId": "CUST001",
	"orderDate": { "$date":
	"2025-01-05T09:30:00Z" },
	"status": "Completed",
	"items": [
	{
	"itemCode": "BV001",
	"name": "Cappuccino",
	"quantity": 2,
	"price": 8.50
	}
	],
	"totalAmount": 17.00,
	"payment": {
	"method": "Cash",
	"paid": true
	}
	}

2.3 Explanation of embedding vs referencing

	Explanation	Example

Embedding	Embedding means storing related data directly inside a document as a sub-document or array	Payment inside orders :
		
		"payment": {
		"method": "Cash",
		"paid": true
		}

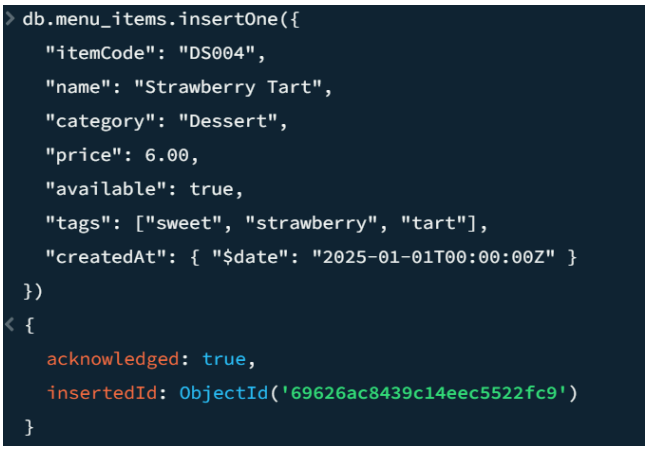
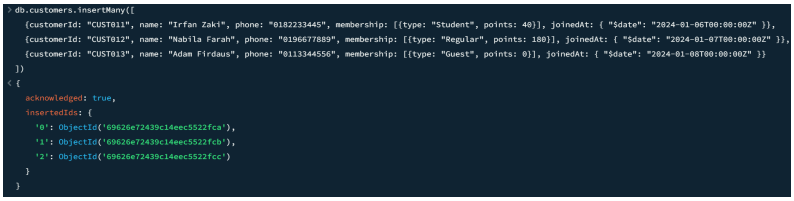
Referencing	Referencing means storing only the ID of another document instead of embedding the full data	Customer ID in orders : "customerId": "CUST001"

Aspect	Embedding	Referencing
Storage	Inside the same document	Stored in another document
Performance	Faster reads	Requires extra lookup
Data Duplication	Possible duplication	Minimal Duplication
Use Case	Tightly related data	Independent data
Scalability	Best for small bounded data	Best for large shared data

2.4 Data types Used

Field	Data Type	Description
String	String	Names, IDs, categories, status
Numeric Value	Double, Int	Prices, quantities, points
Boolean	Boolean	Availability, payment status
Date	Date	Order date, join date, created date
Array	Array	Tags, order items
Embedded document	Object	Membership, payment, order items

3.0 Basic MongoDB Operations (CRUD)

Operation	Output
1. Insert Operations	
• insertOne()	
• insertMany()	
2. Query Operations	

- find()

```
> db.menu_items.find()
< {
    _id: ObjectId('695934de4b77a79e2d6450cd'),
    itemCode: 'BV001',
    name: 'Cappuccino',
    category: 'Beverage',
    price: 8.5,
    available: true,
    tags: [
      'coffee',
      'hot',
      'cold'
    ],
    createdAt: 2025-01-01T00:00:00.000Z
  }
  {
    _id: ObjectId('695934de4b77a79e2d6450ce'),
    itemCode: 'BV002',
    name: 'Latte',
    category: 'Beverage',
    price: 9,
    available: true,
    tags: [
      'coffee',
      'hot'
    ],
    createdAt: 2025-01-02T00:00:00.000Z
  }
  {
    _id: ObjectId('695934de4b77a79e2d6450cf'),
    itemCode: 'BV003',
    name: 'Iced Lemon Tea',
    category: 'Beverage',
    price: 6.5,
```

- findOne()

```
> db.menu_items.findOne({itemCode: "DS004"})
< {
    _id: ObjectId('69626ac8439c14eec5522fc9'),
    itemCode: 'DS004',
    name: 'Strawberry Tart',
    category: 'Dessert',
    price: 6,
    available: true,
    tags: [
      'sweet',
      'strawberry',
      'tart'
    ],
    createdAt: 2025-01-01T00:00:00.000Z
  }
```

- Condition filters using $gt, $lt, $eq, $in, $and, $or

- Find all menu items in the **Dessert** category

```
> db.menu_items.find({
    category: {$eq: "Dessert"}
  }).limit(2)
< {
    _id: ObjectId('695934de4b77a79e2d6450d4'),
    itemCode: 'DS001',
    name: 'Chocolate Cake',
    category: 'Dessert',
    price: 10,
    available: true,
    tags: [
      'sweet',
      'chocolate'
    ],
    createdAt: 2025-01-01T00:00:00.000Z
  }
  {
    _id: ObjectId('695934de4b77a79e2d6450d5'),
    itemCode: 'DS002',
    name: 'Cheesecake',
    category: 'Dessert',
    price: 11,
    available: true,
    tags: [
      'sweet',
      'cheese'
    ],
    createdAt: 2025-01-02T00:00:00.000Z
  }
```

- Find all customers with **more than 100 membership points**

```
> db.customers.find({
    "membership.points": {$gt:100}
  }).limit(2)
< {
    _id: ObjectId('6959368f4b77a79e2d6450d9'),
    customerId: 'CUST001',
    name: 'Aiman Hakim',
    phone: '0123456789',
    membership: {
      type: 'Student',
      points: 120
    },
    joinedAt: 2024-12-01T00:00:00.000Z
  }
  {
    _id: ObjectId('6959368f4b77a79e2d6450dc'),
    customerId: 'CUST004',
    name: 'Siti Aminah',
    phone: '0165566778',
    membership: {
      type: 'Regular',
      points: 200
    },
    joinedAt: 2024-11-20T00:00:00.000Z
  }
```

- Find all menu items that cost less than RM10

```
> db.menu_items.find({
    price: {$lt:10}
  }).limit(2)
< {
    _id: ObjectId('695934de4b77a79e2d6450cd'),
    itemCode: 'BV001',
    name: 'Cappuccino',
    category: 'Beverage',
    price: 8.5,
    available: true,
    tags: [
      'coffee',
      'hot',
      'cold'
    ],
    createdAt: 2025-01-01T00:00:00.000Z
  }
  {
    _id: ObjectId('695934de4b77a79e2d6450ce'),
    itemCode: 'BV002',
    name: 'Latte',
    category: 'Beverage',
    price: 9,
    available: true,
    tags: [
      'coffee',
      'hot'
    ],
    createdAt: 2025-01-02T00:00:00.000Z
  }
```

• Find menu items that are either Beverage or Dessert	

• Find available beverages that cost less than RM9	

```
> db.menu_items.find({
    $and: [
      {category: "Beverage"},
      {available: true},
      {price: {$lt:9}}
    ]
  })
< {
    _id: ObjectId('695934de4b77a79e2d6450cd'),
    itemCode: 'BV001',
    name: 'Cappuccino',
    category: 'Beverage',
    price: 8.5,
    available: true,
    tags: [
      'coffee',
      'hot',
      'cold'
    ],
    createdAt: 2025-01-01T00:00:00.000Z
  }
  {
    _id: ObjectId('695934de4b77a79e2d6450cf'),
    itemCode: 'BV003',
    name: 'Iced Lemon Tea',
    category: 'Beverage',
    price: 6.5,
    available: true,
    tags: [
      'cold',
      'tea'
    ],
    createdAt: 2025-01-03T00:00:00.000Z
  }
```

- Find orders that are either Pending or Failed

```
> db.orders.find({
    $or: [
      { status: "Pending" },
      { status: "FAILED" }
    ]
  }).limit(1)
< {
    _id: ObjectId('695937a14b77a79e2d6450e9'),
    orderId: 'ORD003',
    customerId: 'CUST003',
    orderDate: 2025-01-05T11:15:00.000Z,
    status: 'Pending',
    items: [
      {
        itemCode: 'BV003',
        name: 'Iced Lemon Tea',
        quantity: 2,
        price: 6.5
      }
    ],
    totalAmount: 13,
    payment: {
      method: 'Cash',
      paid: false
    }
  }
```

- Array queries
  - Find all menu items that have the tag **"coffee"**

```
> db.menu_items.find({
    tags: "coffee"
  }).limit(2)
< {
    _id: ObjectId('695934de4b77a79e2d6450cd'),
    itemCode: 'BV001',
    name: 'Cappuccino',
    category: 'Beverage',
    price: 8.5,
    available: true,
    tags: [
      'coffee',
      'hot',
      'cold'
    ],
    createdAt: 2025-01-01T00:00:00.000Z
  }
  {
    _id: ObjectId('695934de4b77a79e2d6450ce'),
    itemCode: 'BV002',
    name: 'Latte',
    category: 'Beverage',
    price: 9,
    available: true,
    tags: [
      'coffee',
      'hot'
    ],
    createdAt: 2025-01-02T00:00:00.000Z
  }
```

Find orders that contain an item where:

- itemCode is "BV001"
- quantity is 2 or more

```
> db.orders.find({
    items: {
      $elemMatch: {
        itemCode: "BV001",
        quantity: { $gte: 2 }
      }
    }
  })
< {
    _id: ObjectId('695937a14b77a79e2d6450e7'),
    orderId: 'ORD001',
    customerId: 'CUST001',
    orderDate: 2025-01-05T09:30:00.000Z,
    status: 'Completed',
    items: [
      {
        itemCode: 'BV001',
        name: 'Cappuccino',
        quantity: 2,
        price: 8.5
      },
      {
        itemCode: 'DS001',
        name: 'Chocolate Cake',
        quantity: 1,
        price: 10
      }
    ],
    totalAmount: 27,
    payment: {
      method: 'Cash',
      paid: true
    }
  }
```

<table>
<tr>
<td>

- Projection (include/exclude fields)
  - Show customer name and membership type only

</td>
<td>

```
> db.customers.find(
    {},
    {
      _id: 0,
      name: 1,
      "membership.type": 1
    }
  ).limit(2)
< {
    name: 'Aiman Hakim',
    membership: {
      type: 'Student'
    }
  }
  {
    name: 'Nur Aisyah',
    membership: {
      type: 'Regular'
    }
  }
```

</td>
</tr>
<tr>
<td>

- Show all customer details except phone number

</td>
<td>

```
> db.customers.find(
    {},
    {
      phone: 0
    }
  ).limit(2)
< {
    _id: ObjectId('6959368f4b77a79e2d6450d9'),
    customerId: 'CUST001',
    name: 'Aiman Hakim',
    membership: {
      type: 'Student',
      points: 120
    },
    joinedAt: 2024-12-01T00:00:00.000Z
  }
  {
    _id: ObjectId('6959368f4b77a79e2d6450da'),
    customerId: 'CUST002',
    name: 'Nur Aisyah',
    membership: {
      type: 'Regular',
      points: 80
    },
    joinedAt: 2024-12-05T00:00:00.000Z
  }
```

</td>
</tr>
<tr>
<td colspan="2">

3. Update Operations
   - updateOne(), updateMany()
   - Update operators:

</td>
</tr>
</table>

<table>
<tr><td colspan="2">- $set, $inc, $push, $pull</td></tr>
<tr><td>

- Update the availability status of the menu item Vegetarian Pasta to available.

</td><td>

```
> db.menu_items.updateOne(
    { name: "Vegetarian Pasta" },
    { $set: { available: true } }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

</td></tr>
<tr><td>

- Increase the membership points of customer CUST001 by 20 points after completing an order

</td><td>

```
> db.customers.updateOne(
    { customerId: "CUST001" },
    { $inc: { "membership.points": 20 } }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

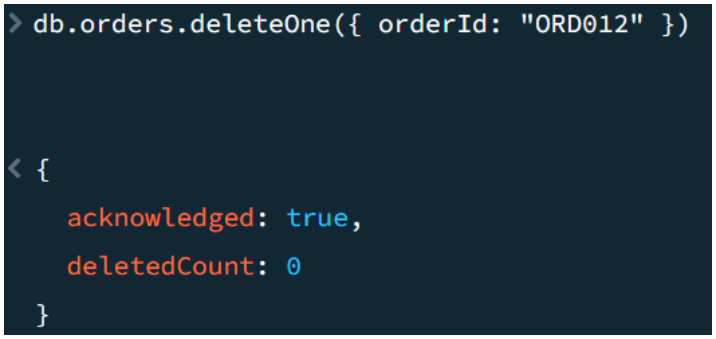
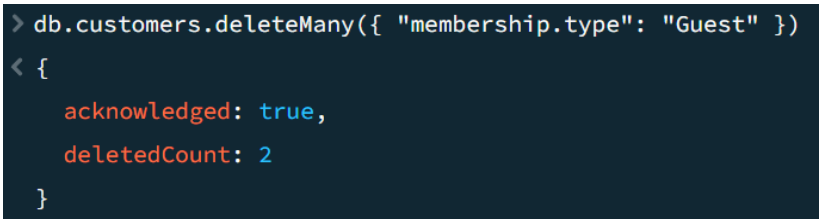
</td></tr>
<tr><td>

- Add a new tag "popular" to the menu item Cappuccino.

</td><td>

```
> db.menu_items.updateOne(
    { name: "Cappuccino" },
    { $push: { tags: "popular" } }
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

</td></tr>
</table>

• Remove the tag **"sweet"** from all menu items where it exists.	
4. Delete Operations	
• deleteOne() - Remove order id: ORDO12	
• deleteMany() - remove guest customer	

4.0 Advanced MongoDB Operations

1. Indexing

- Single-field index
- *Status* is frequently used. So, it is chosen to improve query performance when filtering orders by their current state. Thus, MongoDB does not need to scan the entire collection to find "**COMPLETED**" status.

```
> db.orders.createIndex({ status: 1 })
< status_1
```

```
> db.orders.find({ status: "Completed" })
< {
    _id: ObjectId('696357d69b69b5e56295cbca'),
    orderId: 'ORD001',
    customerId: 'CUST001',
    orderDate: 2025-01-05T09:30:00.000Z,
    status: 'Completed',
    items: [
      {
        itemCode: 'BV001',
        name: 'Cappuccino',
        quantity: 2,
        price: 8.5
      },
      {
        itemCode: 'DS001',
        name: 'Chocolate Cake',
        quantity: 1,
        price: 10
      }
    ],
    totalAmount: 27,
    payment: {
      method: 'Cash',
      paid: true
    }
```

- Compound Index
- The compound index on *customerId* and *orderDate* is created because queries often retrieve orders for a

```
> db.orders.createIndex({ customerId: 1, orderDate: -1})
< customerId_1_orderDate_-1
```

specific customer and sort them by order date. Indexing these fields together optimizes both filtering and sorting operations, resulting in faster access to customer order history.

```
> db.orders.find({ customerId: "CUST001" }).sort({ orderDate: -1 })
< {
    _id: ObjectId('696357d69b69b5e56295cbca'),
    orderId: 'ORD001',
    customerId: 'CUST001',
    orderDate: 2025-01-05T09:30:00.000Z,
    status: 'Completed',
    items: [
      {
        itemCode: 'BV001',
        name: 'Cappuccino',
        quantity: 2,
        price: 8.5
      },
      {
        itemCode: 'DS001',
        name: 'Chocolate Cake',
        quantity: 1,
        price: 10
      }
    ],
    totalAmount: 27,
    payment: {
      method: 'Cash',
      paid: true
    }
  }
```

2. Aggregation pipeline

- using the $group and $project
- this show the total count for the order

```
db.orders.aggregate([
  { $group: { _id: null, totalOrder: { $sum: 1 } } },
  { $project: { _id: 0, totalOrder: 1 } }
])
< {
  totalOrder: 10
}
project2>
```

- using the $group and $project
- to group the list and count the total for each list

```
> db.menu_items.aggregate([
    {$group: {_id: "$category",totalItems: { $sum: 1 }}},
    {$project: {_id: 0,category: "$_id",totalItems: 1}}
  ])
< {
    totalItems: 3,
    category: 'Dessert'
  }
  {
    totalItems: 4,
    category: 'Beverage'
  }
  {
    totalItems: 3,
    category: 'Food'
  }
```

3. Sorting

- sorting menu item price in ascending

```
> db.menu_items.find({}, { _id: 0, itemCode: 1, name: 1, price: 1 })
        .sort({ price: 1 })
        .limit(10)
        .pretty()
< {
    itemCode: 'BV003',
    name: 'Iced Lemon Tea',
    price: 6.5
  }
  {
    itemCode: 'BV004',
    name: 'Chocolate Milk',
    price: 7
  }
  {
    itemCode: 'BV001',
    name: 'Cappuccino',
    price: 8.5
  }
  {
    itemCode: 'BV002',
    name: 'Latte',
    price: 9
  }
  {
    itemCode: 'DS003',
    name: 'Ice Cream Sundae',
    price: 9.5
  }
  {
    itemCode: 'DS001',
    name: 'Chocolate Cake',
    price: 10
  }
  {
    itemCode: 'DS002',
    name: 'Cheesecake',
    price: 11
  }
  {
    itemCode: 'FD001',
    name: 'Chicken Sandwich',
    price: 12
  }
  {
```

- Sort menu item prices in descending order

```
> db.menu_items.find({}, { _id: 0, itemCode: 1, name: 1, price: 1 })
        .sort({ price: -1 })
        .limit(10)
        .pretty()
< {
    itemCode: 'FD002',
    name: 'Beef Burger',
    price: 15.5
  }
  {
    itemCode: 'FD003',
    name: 'Vegetarian Pasta',
    price: 13
  }
  {
    itemCode: 'FD001',
    name: 'Chicken Sandwich',
    price: 12
  }
  {
    itemCode: 'DS002',
    name: 'Cheesecake',
    price: 11
  }
  {
    itemCode: 'DS001',
    name: 'Chocolate Cake',
    price: 10
  }
  {
    itemCode: 'DS003',
    name: 'Ice Cream Sundae',
    price: 9.5
  }
  {
    itemCode: 'BV002',
    name: 'Latte',
    price: 9
  }
  {
    itemCode: 'BV001',
    name: 'Cappuccino',
    price: 8.5
  }
  {
```

4.1 Query Performance Discussion

Indexes significantly improve query performance in MongoDB. It reduces the number of documents that must be scanned to satisfy a query. Without an index, MongoDB performs a full collection scan, examining every document to find matching results. This method becomes inefficient as the dataset grows. When an index is present, MongoDB can quickly locate needed documents using the indexed fields, allowing queries to execute faster and more efficiently. In this project, indexes on frequently queried fields such as *status*, *customerId*, and *orderDate* enable MongoDB to retrieve order data without scanning the entire orders collection. On the other hand, Compound Indexes enhance performance by supporting queries that filter and sort data simultaneously. Although indexes require additional storage, the improvement in read performance makes them essential for optimizing query speed in the café system.

4.2 Challenges encountered

- Choices for theme of the project.
- To gain data for each collection takes time because less data show less output if using *find()* or *sort()*.
- Designing effective compound indexes requires a strict rule. An incorrect sequence of fields can render an index ineffective for specific query patterns.a

5.0 Security & Limitations

Basic access controls in MongoDB can be implemented using authentication and role-based authorization to restrict database operations. For this project, admin has access to manage menu items and view reports. For staff, only can create and update the orders. This helps prevent unauthorized access and accidental data modification.

Injection risks may occur if user input is not properly validated before being used in database queries. In MongoDB, NoSQL injection can happen when untrusted input is directly incorporated into query objects. This risk can be mitigated by validating input data, avoiding dynamic query construction, and using application-level safeguards.

MongoDB also has certain database limitations compared to relational systems. It does not enforce foreign key constraints or strict schemas by default. It means referential integrity must be managed at the application level. The careful design is still required despite having flexibility and scalability. This is to ensure data consistency.

6.0 Teamwork

1. Muhammad Syahmi Faris bin Rusli **- CRUD & Query Operations Lead**

My main role in this project was handling CRUD and query operations using MongoDB. I implemented insert operations to populate the collections with sample data, followed by query operations using find() and findOne() with various condition filters such as comparison and logical operations such as $gt and $in. I also worked on update operations using operators like $set, $push, $pull and $inc, as well as delete operations for data management. In addition, I tested projections and array queries, verified the outputs, and captured the execution results for documentation and presentation.

2. Muhammad Naim Bin Abdullah (free rider on top)- **Data Modelling & Domain Lead**

As data modelling and Domain Lead, I was responsible for defining the project domain and designing the data model based on the theme that we choose which is real world cafe operations. I identified 3 main entities that are required for the system which are customers, menu items, and orders, and translated these requirements into an effective MongoDB document based structure. This also included deciding when to embed data or when to use references, ensuring the design followed MongoDB best practices. I also do part of the Advance operation which is to sort the menu item in ascending and descending order.

3. Ahmad Adib Zikri bin A.Mazlam - **Indexing, Aggregation & Query Performance Lead**

My most contribution to this project is on indexing, aggregation query performance. I create the indexing field using both single-field index and compound index that focus on *status*, *customerId* and *orderDate*. Meanwhile, to perform aggregation, I use $group and $project to show the total count for the order and group the list and count the total for each list. Lastly, I briefly explain how indexing affects query performance. It is by allowing MongoDB to quickly locate relevant documents without performing a full collection scan. Properly designed indexes also help optimize sorting operations, making data retrieval faster and more efficient.

4. Muhammad Mukhritz Al Iman bin Mohd Raffi **- Documentation & Presentation Lead**

As the documentation and presentation lead, I was responsible for writing and organizing the project documentation so that it was clear and easy to understand. I explained the system structure, database design, and the reasons behind our design choices in a simple way. I also do Advance operation which is an Aggregation pipeline where I use $group and $project to show how aggregation works. In addition, I helped prepare the presentation slides and made sure the content was clear, well arranged, and easy to explain during the presentation. This role helped ensure the project was properly documented and presented clearly to others

7.0 Conclusion

Throughout this project, we got a chance to experience designing and implementing a NoSQL database using MongoDB for a cafe ordering system. We learned how NoSQL is different from traditional relational databases. Instead of using fixed tables and foreign keys, MongoDB stores data in documents, allowing related information to be embedded or referenced based on application needs. We also gained practical experience in performing CRUD operations, creating indexes, utilizing aggregate pipelines, and applying sorting. However, some challenges come across while completing this project. At the beginning to choose which system would suit this project. As known, we choose cafe themes to be different from others. Plus, to gain data for each collection takes time because less data show less output if using *find()* or *sort()*. Other than that, designing effective compound indexes requires a strict rule. An incorrect sequence of fields can render an index ineffective for specific query patterns. For the others, there is no such challenge that we faced.

So, the project enhanced our understanding of NoSQL database concepts and encouraged critical thinking in database design. Overall, this project strengthened our ability to apply MongoDB effectively in real-world scenarios especially in cafe ordering systems. Plus, it provided a solid foundation for working with scalable, document-based database systems in future applications.